

ARIA — Multi-Accent Voice Testing Agent

Author: Madhav Raj [AEH Intern @Accenture]

Date: 22-07-2026

GitHub: <https://github.com/madhav14185121/Multi-Accent-Voice-Testing-Agent/>



Introduction

ARIA (Multi-Accent Voice Testing Agent) is an advanced, real-time conversational AI system designed to test and analyze voice interactions across various English accents. With a sleek, premium Next.js frontend and a highly modular FastAPI backend, ARIA provides an end-to-end pipeline from speech-to-text (STT) to large language model (LLM) generation and text-to-speech (TTS) synthesis, along with comprehensive machine learning-based accent detection capabilities.

Problem Statement & Impact

The Problem: Traditional voice agents often struggle with diverse accents, leading to misinterpretations and poor user experiences. Testing these agents requires robust tools that can evaluate accent recognition accuracy, measure latency, and provide clear telemetry.

Impact & Use Cases: ARIA provides a unified interface for testing voice models against different accents (American, British, Indian, Australian, Canadian). It enables developers and QA engineers to continuously evaluate STT accuracy, analyze LLM response relevance, and measure end-to-end latency.

Target Users: AI Researchers, AI Engineers, and Voice Application Developers.

Architecture

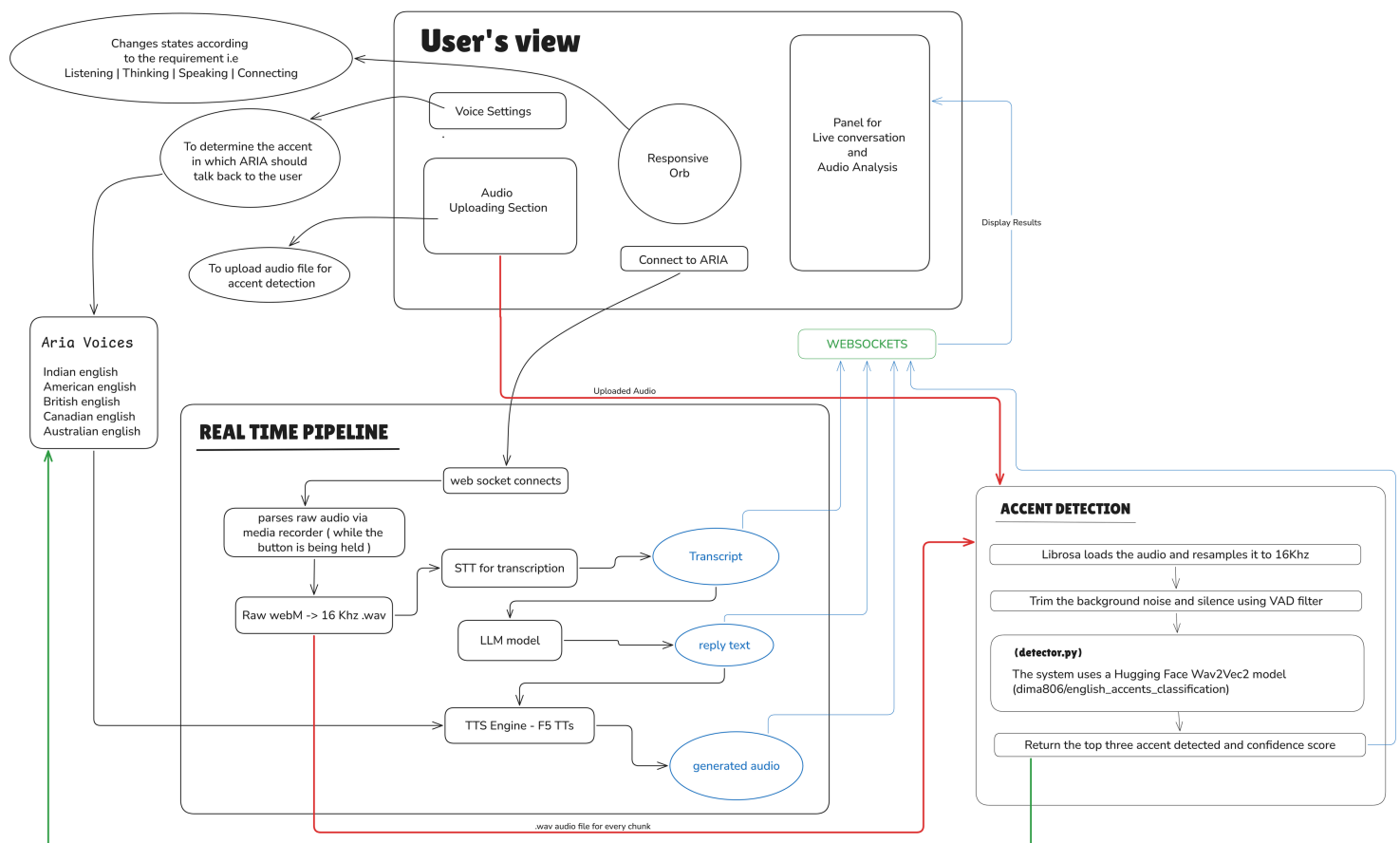


Figure: ARIA - System Architecture

Architecture Flow: The user interacts with the ARIA frontend via a Push-to-Talk interface or audio file upload. The frontend streams audio chunks over WebSockets to the FastAPI backend. The backend orchestrates a modular AI pipeline: Faster-Whisper transcribes the audio (STT), Ollama generates a response (LLM), and F5-TTS synthesizes the reply audio (TTS). For uploaded files, a PyTorch Wav2Vec2 model detects the user's accent and saves a detailed telemetry report in Supabase.

Frontend

Framework: Next.js with React and Tailwind CSS.

File-by-File Breakdown

- **frontend/src/app/page.tsx:** Main entry point and WebSocket orchestrator. Manages MediaRecorder, handles the Push-to-Talk (PTT) interaction, and coordinates the 3-column UI layout.
- **frontend/src/app/globals.css:** Global stylesheet implementing the premium light-themed UI, subtle noise textures, and CSS variables.
- **frontend/src/app/layout.tsx:** Next.js root layout.
- **frontend/src/app/history/page.tsx & [id]/page.tsx:** History browser pages that fetch and display past Supabase telemetry reports.
- **frontend/src/components/HomeLayout.tsx:** Manages the overarching 3-column responsive layout orchestration.
- **frontend/src/components/Orb.tsx:** Advanced 2D Canvas and Framer Motion particle orb that visualizes voice states.
- **frontend/src/components/ControlSidebar.tsx:** Unified container for Voice Selector and Audio Upload settings.
- **frontend/src/components/RightPanel.tsx:** Glassmorphic container dynamically swapping between Live Conversation and Audio Analysis views.
- **frontend/src/components/ConversationPanel.tsx:** Handles ChatGPT-style typewriter effects and auto-scrolling for ARIA's replies.
- **frontend/src/components/LiveAnalysisPanel.tsx:** Renders real-time turn pipeline telemetry and latency stats.
- **frontend/src/components/PipelineStage.tsx:** UI component for tracking individual backend pipeline stages (STT, LLM, TTS).
- **frontend/src/components/AudioUpload.tsx:** Drag-and-drop component for uploading .wav, .mp3, and .m4a files for accent detection.
- **frontend/src/components/VoiceSelector.tsx:** Custom dropdown for choosing ARIA's target accent.

- `frontend/src/components/HistoryDrawer.tsx` : Slide-out drawer for quick access to previous reports.
- `frontend/src/hooks/usePipelineState.ts`: Reusable state machine hook mapping WebSocket backend events to frontend pipeline UI states.
- `frontend/src/lib/api.ts & audio.ts`: Helper utilities for playing Base64 audio and managing API base URLs.

Responsive Orb

The Orb.tsx component is an advanced, lightweight 2D Canvas implementation intertwined with Framer Motion springs.

- **Visual States:** idle (slow breathing), connecting (forming ring), listening (ripples based on mic volume), thinking (rapid pulsing), speaking (expanding flow), and error.
- **State Control:** Transitions are managed via a custom internal createStateMix function using linear interpolation, controlled by the orbState variable in page.tsx.
- **Mapping to Events:** Backend WebSocket events (listening, thinking, speaking, idle) directly trigger these states.

Frontend ↔ Backend Communication

- **Files Handling API:** `page.tsx` manages WebSocket streaming; `AudioUpload.tsx` handles REST API calls.
- **Endpoints Consumed:**
 - `ws://<backend>/ws/audio`: Real-time bidirectional WebSocket for Push-to-Talk streaming.
 - `POST /api/upload`: REST endpoint receiving FormData for accent detection.
 - `GET /api/history` & `DELETE /api/history/{id}`: REST endpoints for report management.
- **Data Formats:** WebM Opus audio chunks, Base64 audio strings, and JSON telemetry payloads.

Backend

Framework: FastAPI with Uvicorn.

File-by-File Breakdown

- **backend/run.py & backend/app/main.py:** Application entry point, router registration, and CORS middleware configuration.
- **backend/app/config.py:** Pydantic Settings class loading environment variables (e.g., Supabase credentials, Ollama models).
- **backend/app/api/websocket.py:** Primary orchestrator for real-time STT -> LLM -> TTS execution via WebSockets (Push-to-Talk async handler).
- **backend/app/api/upload.py:** Handles file uploads, calls the accent detector, and saves reports to Supabase.
- **backend/app/api/history.py :** CRUD REST routes for fetching and deleting historical telemetry reports.
- **backend/app/services/audio/processor.py:** Uses FFmpeg to convert incoming WebM/Opus audio chunks into normalized 16kHz WAV format.
- **backend/app/services/audio/context.py & accumulator.py:** Manages conversational context and buffers speech for analysis.
- **backend/app/services/accent/detector.py:** Hugging Face pipeline wrapping a `dima806/english_accents_classification Wav2Vec2` model for accent detection.
- **backend/app/services/stt/transcriber.py:** Faster-Whisper (`distil-small.en`) singleton for rapid speech-to-text conversion.
- **backend/app/services/llm/client.py:** Dedicated Ollama client for generating conversational responses.
- **backend/app/services/tts/f5_provider.py & engine.py:** Integration with F5-TTS for low-latency, zero-shot voice cloning and synthesis.
- **backend/app/services/storage/supabase_client.py:** Supabase SDK wrapper managing audio-uploads bucket and accent_reports PostgreSQL table.
- **backend/app/services/session/manager.py:** Tracks multi-turn conversation history and context per WebSocket session.

End-to-End Pipeline

1. **Audio Ingestion:** Audio streamed via WebSockets or REST file upload.
AudioProcessor normalizes the audio via FFmpeg.
2. **Accent Detection:** AccentDetector uses Wav2Vec2 to extract accent confidence scores.
3. **STT:** WhisperTranscriber (Faster-Whisper) converts normalized WAV into text.
4. **LLM:** LLMClient sends the transcript to Ollama (Llama 3) to generate a text reply.
5. **TTS:** F5TTSProvider synthesizes the text reply back into speech using the chosen target voice reference.
6. **Response:** Synthesized audio is Base64-encoded and sent back to the frontend over the WebSocket.

Supabase Integration

- **Storage:** Audio files are uploaded to the audio-uploads bucket with unique UUID filenames via supabase_client.py.
- **Database:** Telemetry rows are inserted into the accent_reports table via upload.py, storing file URL, duration, predicted accent, confidence scores, and raw JSON telemetry data.

Customization Guide

Below is the customization guide for the project. It mentions what files to be modified for what change and the specific functions to be altered with. It also mentions the possible alternatives to the current software.

Customization	File(s) to Change	Function(s) / Config to Modify	Notes
Swap LLM model (Ollama)	<code>backend/app/services/llm/client.py</code> , <code>backend/app/config.py</code>	<code>chat(self, messages, stream),</code> <code>settings.ollama_model</code>	Change model name variable or replace Ollama API call with OpenAI/Anthropic SDK.

Customization	File(s) to Change	Function(s) / Config to Modify	Notes
Swap STT model (Whisper → other)	backend/app/services/stt/transcriber.py	<code>_ensure_model()</code> , <code>transcribe(self, audio_path)</code>	Replace Faster-Whisper logic with Deepgram/Google STT API calls.
Swap TTS model (F5-TTS → other)	backend/app/services/tts/f5_provider.py	<code>synthesize(self, text, speaker_wav, language)</code>	Replace F5-TTS inference with ElevenLabs or Coqui API.
Enable GPU usage	backend/app/services/stt/transcriber.py , backend/app/services/tts/f5_provider.py	<code>_detect_device()</code>	Automatically checks <code>torch.cuda.is_available()</code> but can be hardcoded to <code>"cuda"</code> .
Change target language/accent (English → other)	backend/app/services/stt/transcriber.py , backend/app/services/accent/detector.py	<code>transcribe()</code> (<code>language="en"</code>), <code>self.classifier</code> model pipeline	Update STT language parameter and load a different Hugging Face accent model.
Swap accent detection model (Wav2Vec2 → other)	backend/app/services/accent/detector.py	<code>__init__()</code> pipeline, <code>detect()</code>	Change <code>"dima806/english_accents_classification"</code> to another Hugging Face model identifier.
Change Supabase schema	backend/app/api/upload.py , backend/app/services/storage/supabase_client.py	<code>upload_audio()</code> , <code>save_report()</code>	Modify the <code>report_data</code> dictionary keys to match new PostgreSQL columns.

Setup & Run Instructions

Prerequisites: Python 3.11+, Node.js 18+, FFmpeg installed and added to PATH, Ollama installed and running locally.

Environment Variables: Create backend/.env based on backend/.env.example:

```
SUPABASE_URL=your_project_url
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
SUPABASE_BUCKET=audio-uploads
OLLAMA_MODEL=llama3
```

Commands to Run Backend:

```
cd backend
python -m pip install -r requirements.txt
python run.py
```

(Runs FastAPI on <http://localhost:8000>)

Commands to Run Frontend:

```
cd frontend
npm install
npm run dev
```

(Runs Next.js on <http://localhost:3000>)

Common Troubleshooting: -

FFmpeg PATH: Ensure ffmpeg is accessible from the command line, otherwise audio normalization will fail with "ffmpeg not found" errors.

Supabase Errors: Ensure the audio-uploads bucket is set to public if required, and the accent_reports table schema exactly matches upload.py.

CORS / Websocket disconnects: Ensure the frontend .env.local or lib/api.ts points to the correct backend IP/Port.

Tech Stack Summary

Technology	Role / Purpose
Next.js (React)	Frontend web framework providing routing (App Router) and SSR.
Tailwind CSS	Utility-first styling framework for rapid UI development.
Framer Motion	Physics-based animation library used for fluid UI transitions and 3D hover effects.
HTML5 Canvas 2D	High-performance particle rendering engine for the voice Orb.
FastAPI	High-performance Python backend framework handling REST and WebSockets.
Faster-Whisper	Lightning-fast Speech-to-Text (STT) inference model.
Ollama (Llama 3)	Local Large Language Model (LLM) serving engine for conversational logic.
F5-TTS	Zero-shot Text-to-Speech synthesis for generating ARIA's voice replies.
Hugging Face / PyTorch	Wav2Vec2 audio classification for accent detection (transformers pipeline).
FFmpeg	Core audio manipulation and chunk normalization tool.
Supabase	Cloud Postgres database (telemetry) and S3-compatible storage (audio files).

Future Improvements / Roadmap

- **Async Streaming TTS:** Begin playing TTS audio on the frontend before the entire response is fully synthesized.
- **Multilingual Support:** Auto-detect spoken language and dynamically switch Whisper/TTS language parameters.

- **ARIA's own voice:** Aggregate different accent samples for different voices of ARIA and dynamically switch the accent.
- **Custom System Prompts:** Allow users to tweak ARIA's personality directly from the frontend sidebar.

Appendix

Project Folder Structure

```
project/
├── backend/
│   ├── app/
│   │   ├── api/ (websocket.py, upload.py, history.py)
│   │   ├── services/
│   │   │   ├── accent/ (detector.py)
│   │   │   ├── audio/ (processor.py, context.py)
│   │   │   ├── llm/ (client.py)
│   │   │   ├── storage/ (supabase_client.py)
│   │   │   ├── stt/ (transcriber.py)
│   │   │   └── tts/ (f5_provider.py)
│   │   ├── requirements.txt
│   │   └── run.py
│   └── frontend/
│       ├── src/
│       │   ├── app/ (page.tsx, globals.css)
│       │   ├── components/ (Orb.tsx, AudioUpload.tsx, RightPanel.tsx)
│       │   ├── hooks/ (usePipelineState.ts)
│       │   └── lib/ (api.ts)
│       └── package.json
└── Documentation.md
```

Glossary:

- **STT (Speech-to-Text):** Converting spoken audio into written text.
- **TTS (Text-to-Speech):** Synthesizing written text back into spoken audio.
- **LLM (Large Language Model):** AI model generating contextual text responses
- **VAD (Voice Activity Detection):** Identify presence or absence of human speech.
- **Wav2Vec2:** A machine learning framework used for audio classification